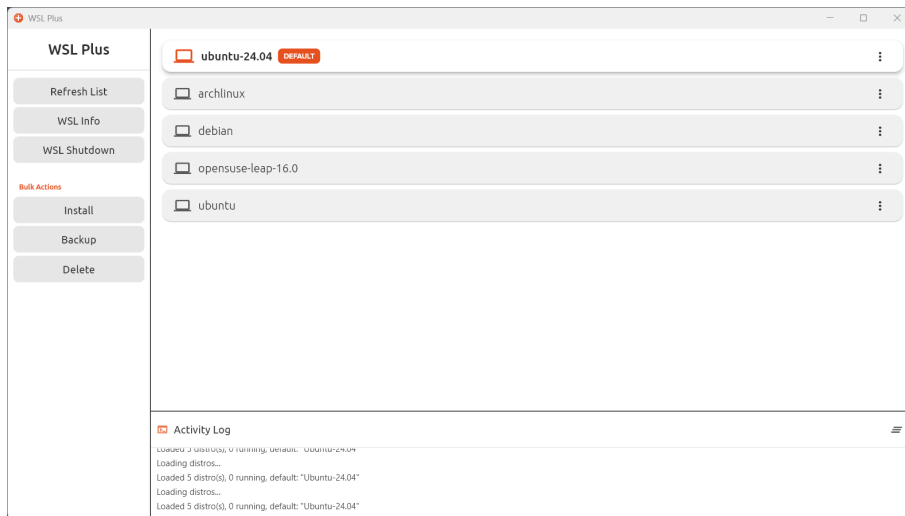




WSL Plus

Management tool for WSL

FINAL REPORT

Name: Shane Crowley
Student number: W20109015
Supervisor: Mary Fitzgerald
Date: 29th March 2026

Lecturer: Colm Dunphy
Module: PROJECT
Course: Higher Diploma in Computer Science

I declare that the work which follows is my own, and that any quotations from any sources (e.g. books, journals, the internet) are clearly identified as such by the use of 'single quotation marks', for shorter excerpt and identified italics for longer quotations. All quotations and paraphrases are accompanied by (date, author) in the text and a fuller citation is the bibliography. I have not submitted the work represented in this report in any other course of study leading to an academic award.

Student.....Shon Crowley..... Date 23-03-26

Work Place Mentor.....NA..... Date NA

Final Report — Shane Crowley

01 April 2026

WSL Plus (wslp) is a tool developed to manage instances of Linux distributions on Windows Subsystem for Linux (WSL) instances. It enables a user to perform a wide set of actions, including installing, deleting, renaming, and backing up through either a CLI or GUI interface. Enhancements are made to the default WSL user experience, including bulk actions, reduced steps, and distro-specific actions. Multiple distros can be installed in a single CLI command, the status of all distros can be monitored simultaneously in a dashboard, and the telemetry features in Ubuntu distros can be toggled with a click.

1. Introduction

1.1. Project overview

The goal of this project was to develop a tool to make using the Windows Subsystem for Linux (WSL) more efficient and convenient.

Public Repo: <https://github.com/edibotopic/wslp/>

The project initially aimed to develop a CLI tool. A stretch goal was to explore how its functionality could be integrated with an [existing Flutter-based GUI app](#) from [Canonical](#) for managing a subscription service. During the project, I instead developed the CLI and a GUI in parallel.

The goal of the project was not to develop a product, but instead to develop a proof-of-concept **open source** tool that could be demoed and used for design ideation.

1.2. Background

This section includes a brief overview of the technologies and tools, which is necessary to understand the motivation behind the project.

1.2.1. WSL

WSL is a virtualization layer built in to recent versions of the Windows OS. It makes it easy and performant to run Linux distributions in a terminal environment.

If WSL is installed and enabled on a Windows machine, users can run the following command to install a distro:

```
wsl --install <distro>
```

The `wsl` refers to `wsl.exe`, the CLI that is installed with WSL.

An instance of a distro is launched with:

```
wsl -d <distro>
```

Contents

- 1. Introduction 3
 - 1.1. Project overview 3
 - 1.2. Background 3
 - 1.3. Problem being addressed 4
 - 1.4. Project objectives 5
- 2. Research and Analysis 5
 - 2.1. Research into existing systems and technologies 5
 - 2.2. Requirements analysis 7
 - 2.3. Technologies used 9
 - 2.4. Feature analysis 9
- 3. Design 10
 - 3.1. System design and architecture 10
 - 3.2. UI design 12
 - 3.3. Methodology 14
- 4. Implementation 18
 - 4.1. Technology stack 18
 - 4.2. Getting started 18
 - 4.3. Development process 19
 - 4.4. Feature summary 38
 - 4.5. Project documentation 38
 - 4.6. Deployment and usage 40
- 5. Testing and Evaluation 40
 - 5.1. General evaluation 40
 - 5.2. Testing approach 40
 - 5.3. Evidence 44
 - 5.4. Evaluation 45
- 6. Reflection 45
 - 6.1. Challenges encountered 45
 - 6.2. Possible future improvements 48
- 7. Acknowledgements 49
 - 7.1. AI and LLM usage 49
- 8. Appendix (confidential) 50
 - 8.1. Corporate and product context 50
 - 8.2. Early implementation discussions within my team .. 51
 - 8.3. Capacity issues mean we cannot develop something like this .. 51
 - 8.4. The impact if my project could be hyper specific 52
 - 8.5. Perspective from gowsl maintainer on semantics 52
- Bibliography 53

To remove an installed instance:

```
wsl --unregister <distro>
```

Ubuntu is the default WSL distribution, so when a user runs `wsl --install` with an argument, Ubuntu is installed. A significant number of Windows machines have Ubuntu installed in this way, and it is a common choice for scientists, engineers, and AI developers who want to run a Linux machine but may work in a Windows-only organization.

1.3. Problem being addressed

1.3.1. Limitations of WSL command line interface

The CLI for WSL on Windows (`wsl.exe`) is limited.

Some actions are tedious multi-step processes.

For example, to **rename** an instance:

- Open Registry Editor (regedit)
- Go to `HCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Lxss`
- Find the subkey for your distro by checking each `DistributionName` value
- Double-click `DistributionName` and enter the new name
- Run `wsl --shutdown` to apply the change

It would be preferable to have something like this:

```
wslp rename <old-name> <new-name>
```

Bulk actions, such as deleting multiple instances, are not possible.

For example, running the following to install multiple instances, will fail:

```
wsl --install Ubuntu-22.04 archlinux  
# FAIL!
```

The same can be said for other common actions, like unregistering distros.

The goal was to develop a tool where these kinds of actions would be possible:

```
wslp install Ubuntu-22.04 archlinux debian  
# Success!  
wslp rename Ubuntu-22.04 Ubuntu-renamed  
# Success!
```

Once the logic for the CLI was implemented, a GUI could then hook into that logic:

```
Check distros to install
=====
```

```
- [X] Ubuntu-22.04
- [X] archlinux
- [ ] debian
```

1.4. Project objectives

The proposed tool was intended to match some of the core functionality of `wsl.exe` but, more importantly, improve it in specific respects:

- New commands
- Bulk actions
- Improved CLI design
- Graphical dashboard

The tool would improve the productivity of WSL power users, including developers and system administrators.

Objectives:

1. Re-implement core functionality from the WSL CLI (list, register)
2. Enhance the functionality of the WSL CLI (bulk actions, concurrency)
3. Demonstrate integration of the business logic with a GUI front-end
4. Enhance the visual appearance and UX of the CLI
5. Auto-generate documentation for the CLI tool for web-hosting

2. Research and Analysis

The initial goal of this project was to improve the CLI experience of WSL users on Windows, thereby enhancing their experience running virtualised Linux environments.

This section briefly examines the technical landscape, user needs, and competitive solutions to inform the design of `wslp`.

2.1. Research into existing systems and technologies

2.1.1. WSL

WSL represents a shift in Microsoft's approach to developer tooling. WSL eliminates the overhead of traditional virtual machines [10]. WSL enables near-native performance for Linux workloads on Windows systems [11].

2.1.2. CLIs

The CLI has evolved significantly since its origins in early computing systems. Modern CLI design prioritizes discoverability, consistency, and user experience alongside functionality [13].

Research into CLI tool design emphasizes several key principles:

- Command consistency and predictability
- Clear, actionable error messages
- Progressive disclosure of complexity
- Scriptability and composability with other tools

2.1.3. Go

The Go programming language has emerged as a popular choice for CLI development due to its single-binary deployment model, cross-compilation capabilities, and strong standard library support for system operations [9].

2.1.4. Market analysis

The target market for wslp consists primarily of developers and system administrators who regularly interact with WSL environments.

2.1.4.1. Target audience

Primary Users

- Software developers using WSL for development workflows
- DevOps engineers managing multiple WSL configurations
- System administrators deploying standardized WSL environments

Secondary Users

- Technical educators introducing Linux to Windows users
- Enterprise IT departments supporting developer workstations

2.1.4.2. Existing solutions

2.1.4.2.1. Native to Windows OS

2.1.4.2.1.1. WSL CLI (`wsl.exe`)

- Strengths: Built-in, no installation required, Microsoft support
- Weaknesses: Limited functionality, poor bulk operation support

2.1.4.2.1.2. PowerShell scripts

- Strengths: Flexible, integrates with Windows ecosystem
- Weaknesses: Technical users only, requires scripting knowledge, no standardization

2.1.4.2.1.3. WSL settings

The new [WSL Settings app](#) is a graphical application for applying global settings that apply to all WSL instances. This tool is available by default on Windows, but it is essentially a wrapper around `.wslconfig` and provides little to no instance management functionality.

2.1.4.2.2. Third party

It is notable that several open source projects have been published recently that offer a graphical dashboard for managing WSL. Judging by GitHub stars alone, which is not the best metric, some of the projects are relatively popular:

- [WSL2 distro manager](#): 3.7 k

- [WSL dashboard](#): 984
- [Easy WSL](#): 579
- [WSL UI](#): 89
- [WSL Commander](#): 18

Closed-source options targeted at enterprise users also exist:

- [Raft WSL](#): NA (closed source)

Comments on some third-party tools

[LxRunOffline](#) provides convenience utilities for WSL management but mainly for managing installed tarballs. Development has stagnated. It's written in C++. Various other one-off scripts and tools exist for managing WSL, but they are fragmented, inconsistent, and unmaintained

One of the most polished and popular is [WSL Distro Manager](#). It offers a graphical interface and generally has parity with the major actions provided by `wsl.exe`. It also supports installing distros from multiple locations and global configuration of WSL. It includes several quality-of-life features, like opening instances in VS Code and running scripts within instances. However, there is no CLI, no bulk actions, and no distro-specific actions. The app also has limited documentation and requires payment to use. Some of its actions also have limitations; for example, renaming an instance only renames (aliases) it within the app itself. In my tests, features like opening the file system of an instance didn't work as expected. It is written in Dart/Flutter and primarily uses subprocess calls to `wsl.exe` for management actions.

2.1.4.3. Market gap

The goal with WSL Plus is to provide a free and open-source CLI and GUI application for managing WSL instances that addresses limitations in the core functionality of `wsl.exe`, such as bulk operations, provides simpler and effective replacement for complex actions, like renaming instances. The aim is to provide tangible improvements in WSL workflows, including reduced steps and increased speed, through a well-documented tool with good UX.

2.2. Requirements analysis

The GoWSL [15] library wraps the `wslApi.dll`, and occasionally the `wsl.exe`, on Windows.

The approach in this project was to interface with WSL through GoWSL, rather than directly sub-processing `wsl.exe`, where possible. The expectation was that this would lead to cleaner code that was easier to extend and test.

When a user runs a command in `wslp`, it would call the WSL API, which will then interact with WSL instances.

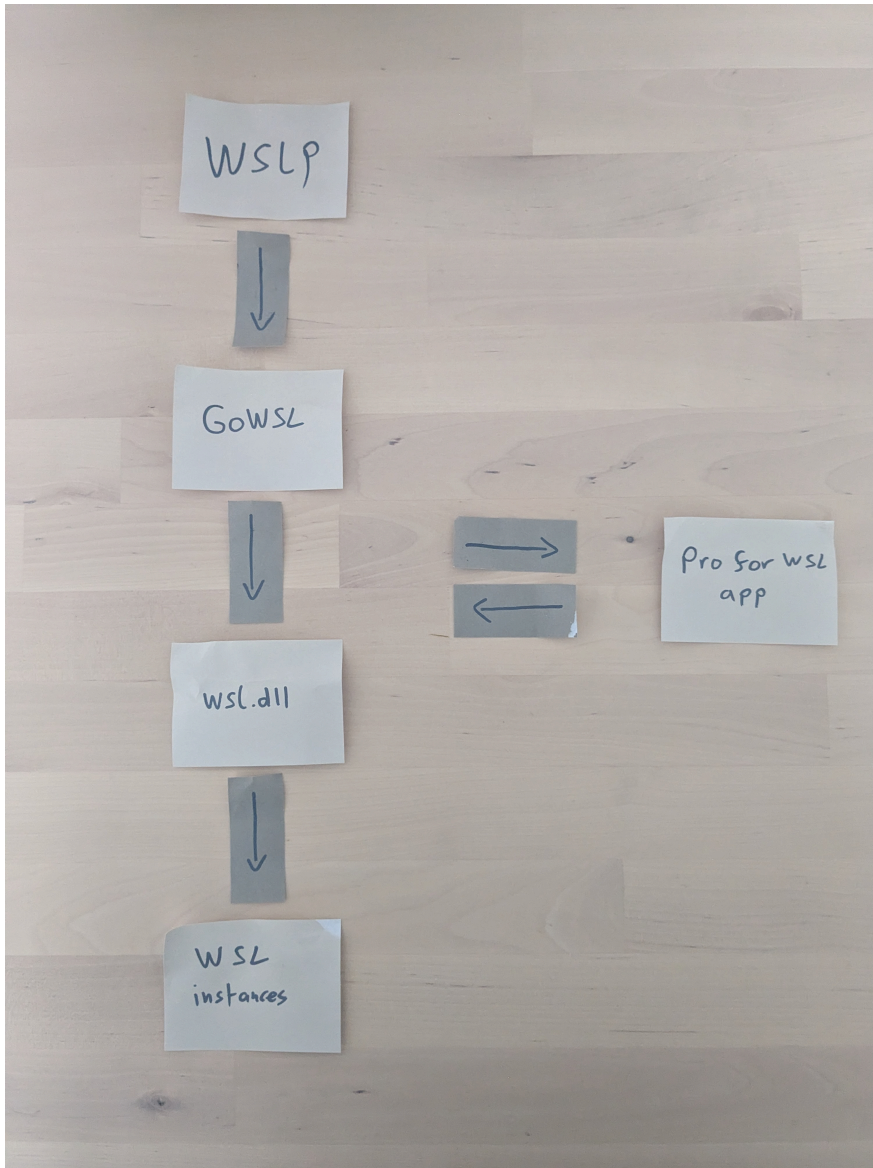


Figure 1: System architecture

Occasionally, it was also necessary to design commands that interacted with the Windows registry.

2.2.1. Front end

There were two front-ends: a Go-based CLI and a Flutter-based GUI.

2.2.2. Back end

The backend architecture leverages the GoWSL library, which provides Go bindings to the Windows Subsystem for Linux API exposed via `wslApi.dll` [15].

Key backend components include:

WSL API Integration

- Distribution management (install, uninstall, terminate)
- Distribution querying (version, state, configuration)

wsl.exe subprocess calls

- When API calls are insufficient

Registry Operations

- Direct manipulation of WSL registry keys when API is insufficient or inefficient (renaming)
- Distro-specific features: toggling telemetry/analytics for Ubuntu instances

2.2.3. Security

There were no major security concerns for the application.

Distros were installed through the official Microsoft repositories.

The app did not touch management of user groups, roles, or credentials.

The security boundary for WSL is shared with the Windows host itself. The major security issues with WSL are (1) a compromised host and (2) the opacity of WSL instances to security tools. The app is independent of these concerns.

2.3. Technologies used

The CLI tool and app backend was developed using:

- The Go programming language [9]
- The Cobra library for CLI design [1]
- The GoWSL library for interfacing with the WSL API [15]
- The Fang library to improve the visual UX of the tool [3]

Go is a well-established language with excellent tooling and an extensive standard library. Its syntax is relatively simple, compared to a language like Rust or C++. It is very commonly used for CLI tools and services, which made it a good fit for this project.

The GUI was developed using Dart and Flutter. Existing applications, like Ubuntu Pro for WSL, have a Go backend and a Flutter front-end. Flutter itself has a relatively low barrier-to-entry and has an excellent developer experience, with features like hot reload. I also liked that Flutter supported publishing both a Windows app and a (limited) browser-based demo, although I never pursued the latter.

Everything was developed on Windows OS with WSL installed.

2.4. Feature analysis

An initial set of features were proposed for `wslp`.

2.4.1. Core features (mvp)

- `list`: Display all installed WSL instances with status
- `install <distro> [distro...]`: Install distribution(s)
- `uninstall <distro> [distro...]`: Remove an installed distribution
- `start <distro>`: Launch a distribution instance

- `stop <distro>` : Terminate a running distribution
- `copy <source> <target>` : Duplicate an existing instance
- `rename <old> <new>` : Change the name of an instance

2.4.2. Stretch features

- Faster installations, leveraging Go's concurrency
- Graphical interface
- Distro-specific actions (Ubuntu telemetry)

2.4.3. Feature prioritization

Features were prioritized based on:

1. User impact (frequency of use, pain points addressed)
2. Implementation complexity (development time, dependencies)
3. Differentiation (added value compared to alternative tools)

3. Design

3.1. System design and architecture

Many tools are built using an architecture in which there is a CLI as well as a GUI, but with a shared business logic defined in isolation from each.

A Go server can have API endpoints that are accessible through the GUI.

The decoupling of the business logic makes the code easier to maintain and test. It also makes it easier to swap out or extend the interfaces if needed.

Originally, I considered writing a CLI that could be interacted with through an existing Flutter app (Ubuntu Pro for WSL), but as the project progressed, I made a standalone GUI to serve that purpose.

3.1.1. Domain model

This section presents the conceptual and technical models that guide the implementation of wslp.

The domain entities included:

3.1.1.1. Core

Distribution (Distro)

- Name: Unique identifier for the instance
- Version: WSL version (1 or 2)
- State: Running, Stopped, Installing, Corrupted

Operation

- Type: Install, uninstall, copy, etc.
- Target: Distribution(s) affected
- Parameters: Command-specific arguments
- Result: Success, failure, partial completion

3.1.1.2. Stretch

Tag

- Name: User-defined label
- Color: Optional visual identifier
- Instances: Set of distributions with this tag

Configuration

- Scope: Global, distribution-specific
- Key-value pairs: Settings and preferences
- Source: Default, user-defined, registry

3.1.2. Data flow model

The initial data flow for the CLI was envisaged as:

```
User Input → CLI Parser → Command Router → Business Logic
Business logic → WSL API / Registry → Validation → User
Output
```

This was later expanded for any integration with the GUI.

3.1.3. System model

The system architecture followed a layered approach. The CLI, for example:

CLI Interface (Cobra)	User interaction layer
Command Handlers (cmd/)	Business logic
GoWSL Library / Registry	System integration
WSL API (wsl.dll)	Windows component

3.1.3.1. Model of GUI implementation (stretch)

- User clicks Flutter button
- Request sent to Go backend
- Action initiated in backend
- GUI status updated in Flutter app

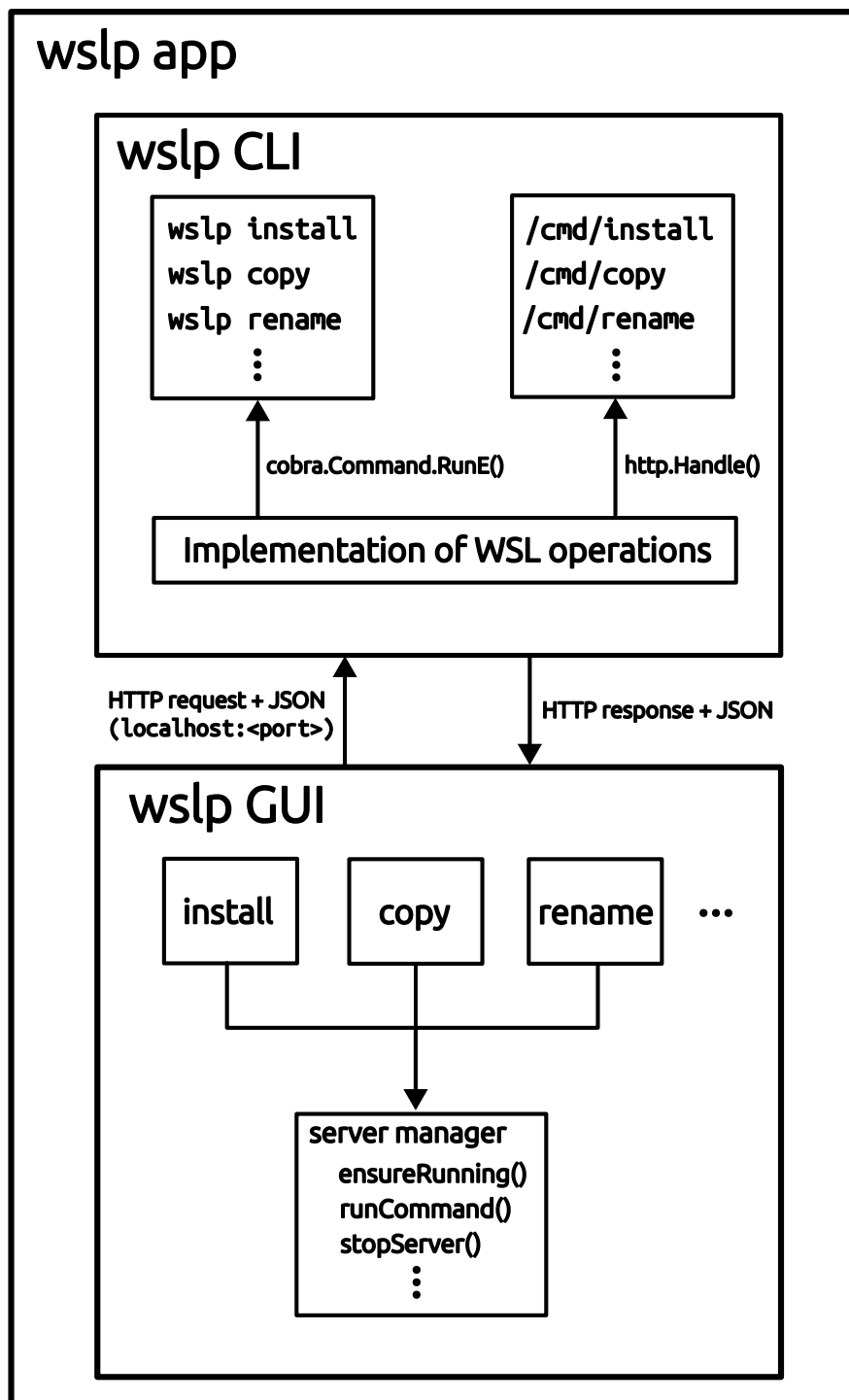


Figure 2: Model of GUI implementation

There is one implementation of the core business logic, which is used by the CLI and the GUI.

3.2. UI design

One of the main UX questions concerned the command hierarchy for the CLI. Generally, it was preferable to keep the hierarchy relatively flat, which was simpler for the user.

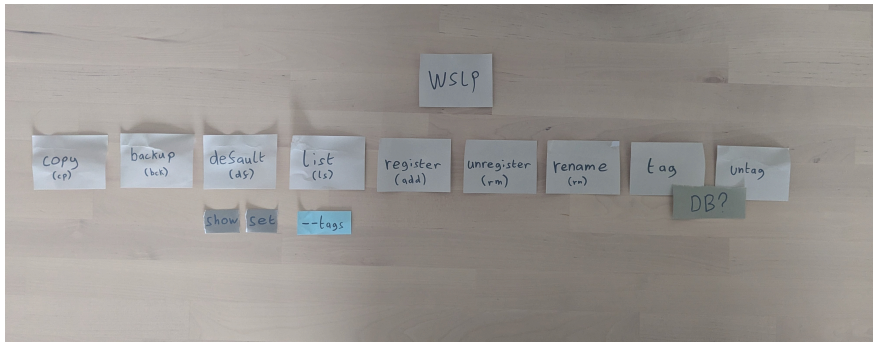


Figure 3: Flat command hierarchy

If the set of commands became sufficiently complex, then it was necessary to establish a layered hierarchy, where users first specify the object they wish to affect:

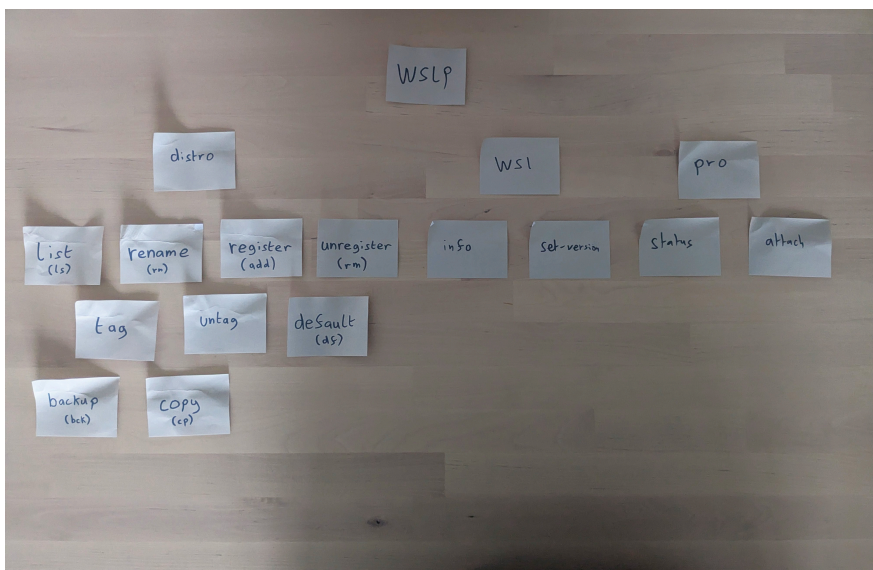


Figure 4: Layered command hierarchy

In either case, the CLI followed the verb-object principle, where a user specifies a verb (`install`) followed by an object (`<distro>`), either at the level of command or sub-command.

For the GUI, Flutter provides a range of premade widgets for building common UI elements, like dialogs, spinners, and grids.

3.2.1. Frontend and UI

The UI interfaces prioritized clarity, discoverability, and consistency.

3.2.1.1. Design principles

Consistency

- All commands follow verb-noun pattern
- Output formatting maintained across all commands

Discoverability

- Built-in help
- Example usage shown for CLI commands
- Error messages suggest correct usage

Feedback

- Progress indicators for operations long operations
- Success confirmations with action taken
- Clear error messages with recovery steps

3.2.1.2. Output formatting

Handled by Fang lib.

3.3. Methodology

3.3.1. Development phases

3.3.1.1. Phase 1: foundation (weeks 1-2)

Objectives

- Set up development environment
- Establish project structure using Cobra
- Implement basic CLI scaffolding
- Create minimal command set (list, install)
- Implement a bulk command
- First meeting with supervisor

Deliverables

- Runnable CLI binary
- Basic help documentation
- Unit test framework

3.3.1.2. Phase 2: core features (weeks 3-4)

Objectives

- Implement core instance management commands
- Add uninstall functionality
- Develop copy and rename operations
- Add automated documentation

Deliverables

- MVP feature set operational
- Integration tests for core workflows
- User documentation for basic commands

3.3.1.3. Phase 3: enhanced operations (weeks 5-6)

Objectives

- Implement tagging system
- Develop configuration management
- Explore GUI integration

Deliverables

- Advanced feature set completed
- Extended documentation

3.3.1.4. Phase 4: polish and report (weeks 6-7)

Objectives

- Comprehensive testing
- Documentation complete

Deliverables

- Production-ready release
- Complete documentation
- Distribution packages

3.3.2. Risk management

Technical Risks

- WSL API changes breaking compatibility
 - Mitigation: API is very stable
- GoWSL library limitations
 - Mitigation: Direct API calls or registry modification where necessary

Schedule Risks

- Feature scope creep
 - Mitigation: Strict MVP definition, backlog prioritization
- Unforeseen technical challenges
 - Mitigation: Descoping of stretch features

Resource Risks

- Single developer
 - Mitigation: Clear documentation, modular architecture for potential contributors

3.3.3. Agile

Sprints were of 2-week duration.

Each task is assigned a point score based on estimated effort.

Major work packages were organized into epics.

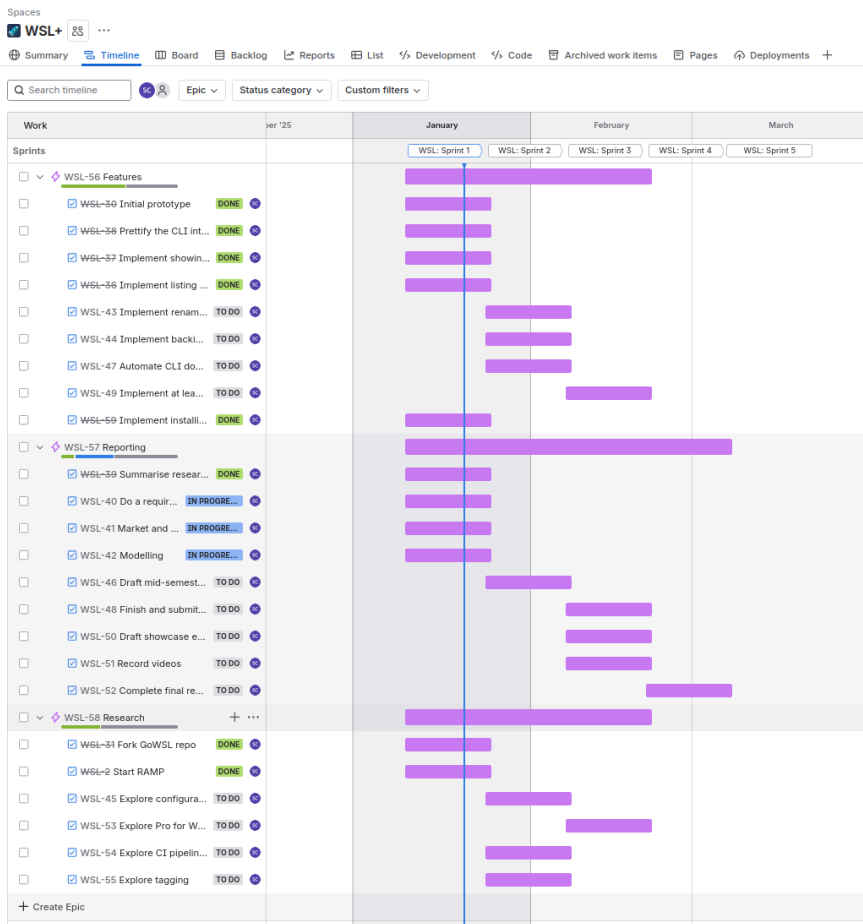


Figure 5: Jira project timeline

Features were implemented and then iterated on using trunk-based development [12]:

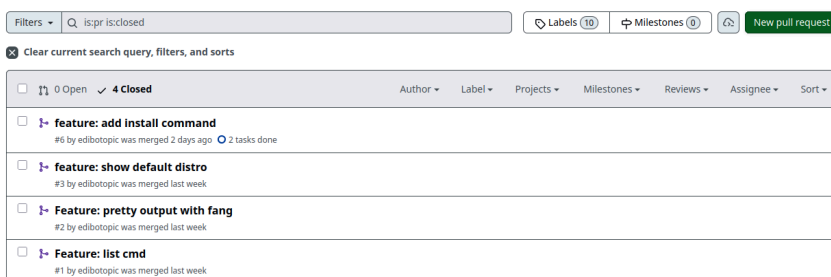


Figure 6: Pull request workflow on GitHub

PRs were raised against one trunk (`main`) and there were no long-lived feature branches.

Experimentation was permitted: if sufficient capacity was available in a given sprint, an unplanned feature was explored.

Bugs and issues were initially captured using the GitHub issue tracker and translated into Jira tickets. But this was abandoned as burdensome for a solo developer.

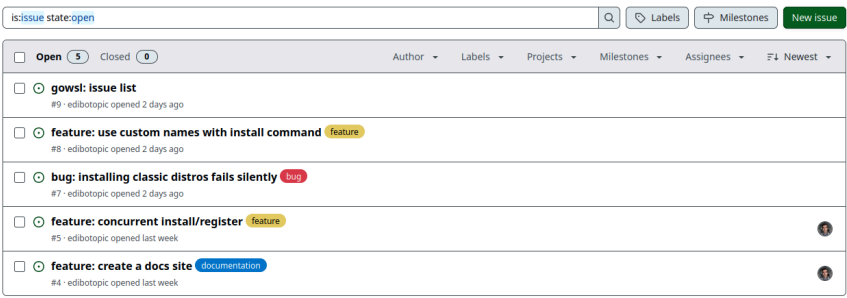


Figure 7: Issue tracking on GitHub

Timeline before final sprint:

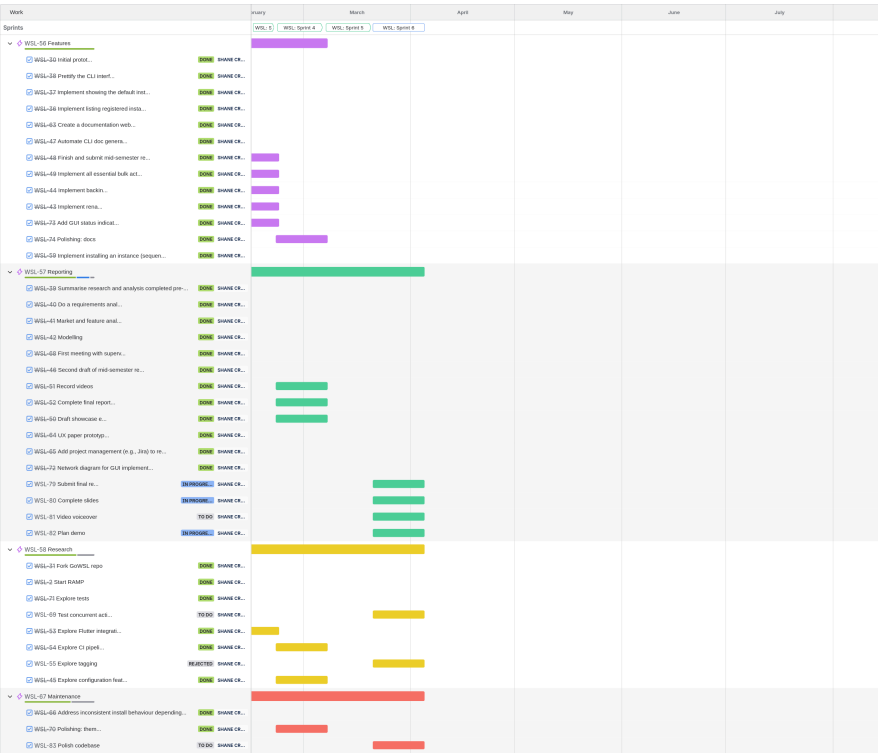


Figure 8: Updated project timeline before final sprint

Cumulative progress during project:

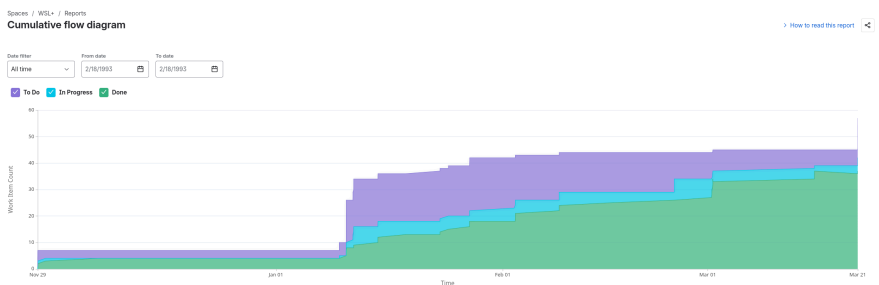


Figure 9: Cumulative progress during the project

The burndown chart for an example sprint is shown below:

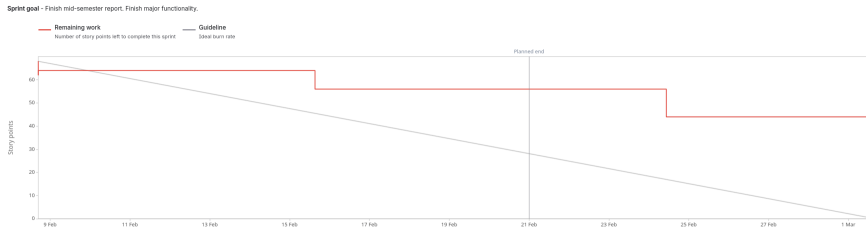


Figure 10: Burndown chart for sprint 3

3.3.4. Tools

- Project management and task tracking: Jira
- Version control: Git
- Hosting: GitHub
- Notes: Confluence (not really used in the end)

4. Implementation

This section documents the technical implementation details of wslp.

4.1. Technology stack

Core Language

- Go 1.21+ for performance, readability, and strong library ecosystem [9]

Key Libraries

- Cobra: CLI framework providing command structure and flag parsing [1]
- GoWSL: WSL API bindings for Go [15]
- Fang: Terminal UI enhancements for improved visual output
- Viper: Configuration file and environment variable management [14] (not well developed)

Development tools

- golsp: code completion and diagnostics
- go test: built in testing tools for Go

4.2. Getting started

4.2.1. Development environment setup

Prerequisites

- Windows 11 with WSL2 installed
- Go 1.21 or later
- Flutter 3.0 or later
- Git for version control
- Code editor of choice

Initial setup

```
# Clone the repository
git clone https://github.com/edibotopic/wslp
```

```
cd wslp

# Install dependencies
go mod download

# Install Cobra CLI tool
go install github.com/spf13/cobra-cli@latest

# Build the project
go build -o wslp.exe

# Run
wslp.exe install <distro1> <distro2>
```

4.2.2. Initial project structure

```
wslp/
├── cmd/           # Cobra command implementations
├── test/          # Integration tests (todo)
├── docs/          # Documentation
├── go.mod         # Module definition
├── go.sum         # Dependency record
└── main.go       # Entry-point
```

4.2.3. Development workflow

1. Identify required feature
2. Create feature branch from `main`
3. Submit pull request with description
4. Merge upon review

4.3. Development process

Summary outline for the development process. Relevant commit hashes are included where possible.

4.3.1. Initializing a Cobra project

[Commit af8910d](#)

I followed the official instructions for setting up a Go environment on my machine:

<https://go.dev/doc/install>

I also installed gopls LSP for my editor to get autosuggestions and code completion:

<https://go.dev/gopls/>

To start a new Go project, I initialized a module:

```
cd ~/dev/ && mkdir wslp && cd wslp
go mod init wslp
```

Cobra is a Go library for making CLI applications.

If you have go installed already, the cobra-cli can be installed system-wide with:

```
go install github.com/spf13/cobra-cli@latest
```

To scaffold a cli, run:

```
cobra-cli init
```

This adds a root command for the cli in `cmd/root.go`:

```
package cmd

import (
    "os"
    "github.com/spf13/cobra"
)

var rootCmd = &cobra.Command{
    Use:   "wsler",
    Short: "A tool for managing WSL instances.",
    Long:  `Create, delete, and rename instances. Perform bulk
actions on multiple instances.`,
}

func Execute() {
    if err := rootCmd.Execute(); err != nil {
        os.Exit(1)
    }
}

func init() {
    // Here you will define your flags and configuration
    settings.
    // Cobra supports persistent flags, which, if defined
    here,
    // will be global for your application.

    // rootCmd.PersistentFlags().StringVar(&cfgFile, "config",
    "", "config file (default is $HOME/.wsler.yaml)")
}
```

It also creates a `main.go` entrypoint, so the Go project can then be run with:

```
go run .
```

Each time you want to add a command to the CLI, run:

```
cobra-cli add <cmd-example>
```

This creates a boilerplate `/cmd/cmd-example.go` file.

You can then test a command with:

```
go run . wslp <cmd-example>
```

4.3.2. Add dependencies

[Commit 2a2e96e](#) (add GoWSL dep)

[Commit 34e6ea5](#) (add Fang dep)

Two dependencies were added early in the project:

- Fang: wrapper for Cobra CLIs to improve UX
- GoWSL: interface to the WSL api on Windows

You add them as dependencies to your project with:

```
go get github.com/charmbracelet/fang
go get github.com/Ubuntu/GoWSL
```

These dependencies need to be added to the relevant files as imports:

```
import (
    "github.com/ubuntu/gowsl"
    ...
    ...
)
```

Cobra is also a dependency, but this was handled with `cobra-cli init`.

When something seems to have gone wrong with your dependencies, run:

```
go mod tidy
```

You can check the active dependencies for the project with `cat go.mod`:

```
toolchain go1.24.7

require (
    github.com/charmbracelet/fang v0.4.4
    github.com/spf13/cobra v1.9.1
    github.com/ubuntu/gowsl v0.0.0-20251112191800-0ef2623cc8fb
)

require (
    charm.land/lipgloss/v2 v2.0.0-
beta.3.0.20251106193318-19329a3e8410 // indirect
    github.com/charmbracelet/colorprofile v0.3.3 // indirect
    ...
    ...
)
```

The dependencies marked `// indirect` are transitive.

4.3.3. Implement the first command

[Commit 0758b7f](#) ([implement list](#))

Here I will break down the first command. Each command followed a similar pattern, and includes similar Cobra boilerplate.

The command I started with was `list`, which recreated `wsl.exe --list`. This command is relatively simple, in that it just gets and prints data.

I wasn't sure at this point, but I thought it might be important later for showing installed distros in a GUI, for example.

4.3.3.1. Scaffold

Use `cobra-cli` to scaffold with default boilerplate.

```
cobra-cli add list
```

4.3.3.2. Code

All the commands are part of the same package `cmd`:

```
package cmd
```

Cobra is needed for the CLI, gowsl for interfacing with WSL. The context and fmt are from the standard library:

```
package cmd

import (
    "context"
    "fmt"

    "github.com/spf13/cobra"

    gowsl "github.com/ubuntu/gowsl"
)
```

Metadata is included with each command to support usage:

```
package cmd

import (
    ...
)

var listCmd = &cobra.Command{
    Use:   "list",
    Short: "List registered WSL distros",
    Long:  `Lists all WSL distributions registered on the
Windows host.`,
    ...
}

...
```

The business logic for the command is explained below with comments:

```
package cmd

import (
    ...
)

var listCmd = &cobra.Command{
    ...
    // The Run field is assigned a function
    // It takes a pointer to the command struct and args from
    the user
    Run: func(cmd *cobra.Command, args []string) {
        // Basic print statement to orient user
        // Dot notation used to call a print function from the
        fmt package
        fmt.Println("Finding registered distros...")
        // RegisteredDistros returns a slice containing the
        registered distros
        // It also returns an error
        // We assign both using := so the compiler can infer the
        types
        distroList, err :=
        gowsl.RegisteredDistros(context.Background())

        // Check that there is an error before proceeding
        if err != nil {
            fmt.Println(err)
        }

        // Print the number of distros by gets the length of the
        slice
        fmt.Println(len(distroList), "distros are registered:")

        // Print the list of distros using a conventional for
        loop
        for i := range distroList {
            fmt.Println(distroList[i].Name())
        }
    },
}
...
```

The last thing to do is add `list` as a subcommand to the root command:

```
...
func init() {
    rootCmd.AddCommand(listCmd)
}
```

We can then run it like this:

```
wslp --help # shows list as an available command
wslp list # runs the list command
```

4.3.4. Make the CLI look good

[Commit 9433706](#) (wrap CLI with Fang)

A simple way to get the CLI looking nice was to wrap the root command in `fang.Execute()`.

This expects a context as arg, so I imported `context` from the standard library too.

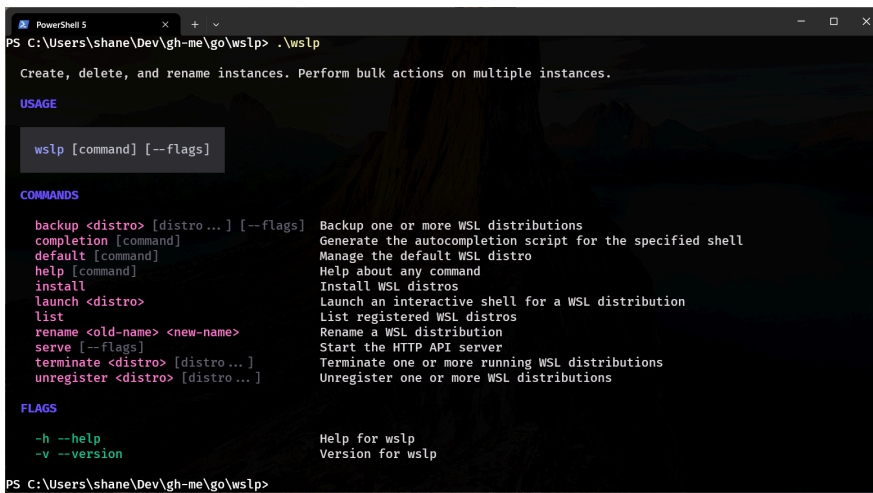
```
package cmd

import (
+  "context"
  "os"

+  "github.com/charmbracelet/fang"
  "github.com/spf13/cobra"
)

func Execute() {
-  if err := rootCmd.Execute(); err != nil {
+  if err := fang.Execute(context.Background(), rootCmd);
err != nil {
    os.Exit(1)
  }
}
```

Example of how the CLI looks after wrapping with Fang:



```
PS C:\Users\shane\Dev\gh-me\go\wslp> .\wslp

Create, delete, and rename instances. Perform bulk actions on multiple instances.

USAGE

wslp [command] [--flags]

COMMANDS

backup <distro> [distro...] [--flags] Backup one or more WSL distributions
completion [command] Generate the autocompletion script for the specified shell
default [command] Manage the default WSL distro
help [command] Help about any command
install Install WSL distros
launch <distro> Launch an interactive shell for a WSL distribution
list List registered WSL distros
rename <old-name> <new-name> Rename a WSL distribution
serve [--flags] Start the HTTP API server
terminate <distro> [distro...] Terminate one or more running WSL distributions
unregister <distro> [distro...] Unregister one or more WSL distributions

FLAGS

-h --help Help for wslp
-v --version Version for wslp

PS C:\Users\shane\Dev\gh-me\go\wslp>
```

Figure 11: CLI appearance after Fang integration

4.3.5. Add a bulk command

[Commit fb2c69](#) (bulk install command)

One of the value propositions with the app was that it enabled bulk WSL actions.

The `wsl.exe` CLI does not allow bulk actions, so the following fails:

```
wsl --unregister Ubuntu Debian
```

The first bulk action that I implemented was for `wslp install`.

NOTE: I am going to focus on the core logic here and not repeat implementation details about Cobra.

4.3.5.1. First implementation

The initial implementation was simple.

The `gowsl` library provided an `Install` function that takes a distro as argument.

If the user passed in one or more distros, the code looped through them, running `gowsl.Install()` each time:

```
var installCmd = &cobra.Command{
    ...
    Run: func(cmd *cobra.Command, args []string) {
        fmt.Println("install called")
        if len(args) == 0 {
            fmt.Println("Error: No distros specified")
            return
        }

        for _, distro := range args {
            fmt.Printf("Installing %s...\n", distro)
            if err := gowsl.Install(context.Background(), distro);
            err != nil {
                fmt.Printf("Error installing %s: %v\n", distro, err)
                continue
            }
            fmt.Printf("Successfully installed %s\n", distro)
        }
    },
}
```

There was nothing fancy here, but it was convenient from a user-perspective.

But, there was an issue, in that the installation behaviour varied across distros in a way that was somewhat opaque to users.

4.3.5.2. Second implementation

[Commit 3ae9226 \(registration check\)](#)

The solution was to add a check to determine if the distro was registered after being installed.

Distros in the modern format get registered on installation while distros in the legacy format do not.

Any necessary instructions could then be printed out for the user:

```

...
var installCmd = &cobra.Command{
    ...
    fmt.Printf("Successfully installed %s\n", distro)

    d := gowsl.NewDistro(context.Background(), distro)

    registered, err := d.IsRegistered()

    if err != nil {
        fmt.Printf("Error checking registration: %v\n", err)
    } else if registered {
        fmt.Printf("%s was downloaded in the modern format
and is already registered\n", distro)
        fmt.Printf("Launch with wsl -d %s", distro)
    } else {
        fmt.Printf("%s was downloaded in the classic format
and must be registered\n", distro)
        fmt.Printf("Register with wsl --register %s",
distro)
    }
    },
},
}

```

4.3.6. Autogenerate CLI docs

Commit [b2dd37a](#) (autogen docs)

I didn't want to hand-write CLI reference documentation each time I added a new command.

4.3.6.1. docgen code

To generate documentation for the CLI tool, I followed the Cobra docs:

<https://cobra.dev/docs/how-to-guides/clis-for-llms/>

```

package main

import (
    "flag"
    "fmt"
    "log"
    "os"
    "path/filepath"
    "strings"

    "github.com/spf13/cobra/doc"
    "wslp/cmd"
)

func main() {
    out := flag.String("out", "../../docs/reference/", "output
directory")
    format := flag.String("format", "markdown", "markdown|man|
rest")

```

```

    front := flag.Bool("frontmatter", false, "prepend simple
YAML front matter to markdown")
    flag.Parse()

    if err := os.MkdirAll(*out, 0o750); err != nil {
        log.Fatal(err)
    }

    rootCmd := cmd.RootCmd
    rootCmd.DisableAutoGenTag = true // stable, reproducible
files (no timestamp footer)

    switch *format {
    case "markdown":
        if *front {
            prep := func(filename string) string {
                base := filepath.Base(filename)
                name := strings.TrimSuffix(base, filepath.Ext(base))
                title := strings.ReplaceAll(name, "_", " ")
                return fmt.Sprintf("---\ntitle: %q\nslug:
%q\ndescription: \"CLI reference for %s\"\n---\n\n", title,
name, title)
            }
            link := func(name string) string { return
strings.ToLower(name) }
            if err := doc.GenMarkdownTreeCustom(rootCmd, *out,
prep, link); err != nil {
                log.Fatal(err)
            }
        } else {
            if err := doc.GenMarkdownTree(rootCmd, *out); err !=
nil {
                log.Fatal(err)
            }
        }
        ...
    default:
        log.Fatalf("unknown format: %s", *format)
    }
}

```

I needed to export `RootCmd` for this to work:

```

- var rootCmd = &cobra.Command{
+ var RootCmd = &cobra.Command{
...

```

Example of an auto-generated CLI reference page:

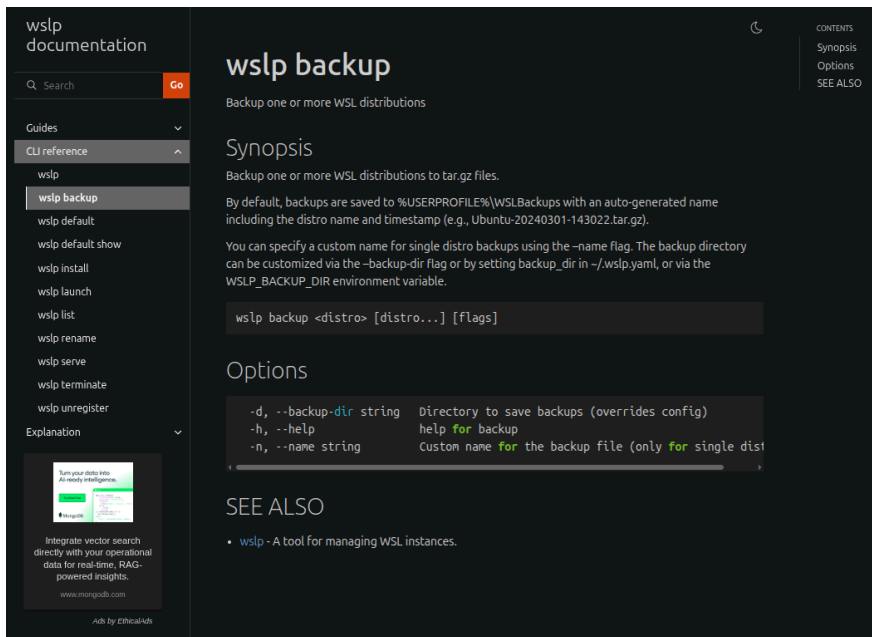


Figure 12: Auto-generated CLI reference page

4.3.6.2. Update makefile

The documentation [6] is built using Sphinx using a starter pack from Canonical.

This starter pack includes a makefile (or equivalent batch file for Windows), which I modified to add a `make cli-ref` target:

```
@echo "* generate CLI reference docs:                               make
cli-ref"

...
.PHONY: help full-help html epub pdf linkcheck spelling
spellcheck woke \
    vale pally run serve install pally-install \
    vale-install pdf-prep pdf-prep-force clean clean-doc
\
    update lint-md cli-ref

...
run: install cli-ref
. $(VENV); $(VENVDIR)/bin/sphinx-autobuild -b dirhtml --host
$(SPHINX_HOST) --port $(SPHINX_PORT) "$(SOURCEDIR)"
"$(BUILDDIR)" $(SPHINXOPTS)
...
cli-ref:
@echo "Generating CLI reference documentation..."
@cd .. && go run ./cmd/docgen -out ./docs/reference -format
markdown
@echo "CLI reference generated in ./reference/"
```

To update the CLI ref:

```
make cli-ref
```

Or, while previewing the docs locally:

```
make run
```

The docs generated are just markdown, so they get added to the published docs.

4.3.7. Refactor logic

[Commit 564c410](#) (refactor log into internal)

4.3.7.1. Moving code

The logic was tied to the CLI implementation.

I began migrating the core logic to `internal/wsl/`.

For example, here is `internal/wsl/list.go`:

```
package wsl

import (
    "context"
    "fmt"

    gowsl "github.com/ubuntu/gowsl"
)

// DistroInfo represents basic information about a WSL
// distro
type DistroInfo struct {
    Name string `json:"name"`
}

// ListDistros retrieves all registered WSL distributions
func ListDistros(ctx context.Context) ([]DistroInfo, error) {
    {
        distroList, err := gowsl.RegisteredDistros(ctx)
        if err != nil {
            return nil, fmt.Errorf("failed to get registered
distros: %w", err)
        }

        result := make([]DistroInfo, len(distroList))
        for i := range distroList {
            result[i] = DistroInfo{
                Name: distroList[i].Name(),
            }
        }

        return result, nil
    }
}
```

4.3.7.2. Calling external functions in commands

Go exports functions that start with a capital letter.

The new `ListDistros()` function in `internal/wsl` could now be called in `cmd/list.go` with:

```

...
func ListDistros(ctx context.Context, l wsl.Listener, out
io.Writer) error {
    fmt.Fprintln(out, "Finding registered distros...")

    distros, err := wsl.ListDistros(ctx, l)
    ...

```

4.3.8. Connect GUI to logic

Commit [f6e525b](#) (server logic)

4.3.8.1. Server

Go makes it easy to write servers, and I started with the basic example for a HTTP server from Go by Example:

<https://gobyexample.com/http-server>

I used a multiplexer for routing multiple requests:

```

// server/server.go
package server

import (
    "context"
    "encoding/json"
    "fmt"
    "net/http"

    "wslp/internal/wsl"
)

type Server struct {
    port string
}

func NewServer(port string) *Server {
    return &Server{
        port: port,
    }
}

func (s *Server) Start() error {
    mux := http.NewServeMux()

    mux.HandleFunc("/api/distros", s.handleListDistros)

    addr := fmt.Sprintf(":%s", s.port)
    fmt.Printf("Starting server on http://localhost%s\n",
    addr)
    return http.ListenAndServe(addr, corsMiddleware(mux))
}

func (s *Server) handleListDistros(w http.ResponseWriter, r
*http.Request) {
    if r.Method != http.MethodGet {
        http.Error(w, "Method not allowed",

```

```

http.StatusMethodNotAllowed)
    return
}

distros, err := wsl.ListDistros(context.Background())
if err != nil {
    http.Error(w, err.Error(),
http.StatusInternalServerError)
    return
}

w.Header().Set("Content-Type", "application/json")
json.NewEncoder(w).Encode(map[string]interface{}{
    "distros": distros,
    "count":   len(distros),
})
}

// to support web-based Flutter deploy
func corsMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r
*http.Request) {
        w.Header().Set("Access-Control-Allow-Origin", "*")
        w.Header().Set("Access-Control-Allow-Methods", "GET,
POST, PUT, DELETE, OPTIONS")
        w.Header().Set("Access-Control-Allow-Headers", "Content-
Type")

        if r.Method == http.MethodOptions {
            w.WriteHeader(http.StatusOK)
            return
        }

        next.ServeHTTP(w, r)
    })
}

```

Note that the list distros handler called the `ListDistros` function from `internal/wsl`.

This server could then be started from the CLI:

```

var serveCmd = &cobra.Command{
    ...
    Run: func(cmd *cobra.Command, args []string) {
        port, _ := cmd.Flags().GetString("port")

        s := server.NewServer(port)
        if err := s.Start(); err != nil {
            fmt.Printf("Server error: %v\n", err)
        }
    },
}

```

As the project progressed, I added more routes and handlers:

```
func (s *Server) Start() error {
    mux := http.NewServeMux()

    // Register routes
    mux.HandleFunc("/api/distros", s.handleListDistros)
    mux.HandleFunc("/api/default", s.handleGetDefault)
    mux.HandleFunc("/api/available", s.handleListAvailable)
    mux.HandleFunc("/api/install", s.handleInstall)
    mux.HandleFunc("/api/unregister", s.handleUnregister)
    mux.HandleFunc("/api/set-default", s.handleSetDefault)
    mux.HandleFunc("/api/backup", s.handleBackup)
    mux.HandleFunc("/api/terminate", s.handleTerminate)
    mux.HandleFunc("/api/launch", s.handleLaunch)
    mux.HandleFunc("/api/rename", s.handleRename)
    mux.HandleFunc("/api/wsl-info", s.handleWSLInfo)
    mux.HandleFunc("/api/distro-info", s.handleDistroInfo)
    ...
    ...
}
```

Initially, to test these routes, I started the server:

```
wslp serve
```

Then checked the returned JSON:

```
curl http://localhost:8080/api/distros
```

Which returned something like:

```
[
  {"name": "Ubuntu-22.04", "running": true},
  {"name": "Debian-11", "running": false}
]
```

4.3.9. Make the GUI do something

[Commit 6cec8d4](#) (add api service for GUI)

To allow GUI interactions to hit the server endpoints, I wrote:

`gui/lib/services/api_service.dart`.

The version below only implements `list`:

```
import 'dart:convert';
import 'package:http/http.dart' as http;

class ApiService {
    // server is hard-coded to port 8080 by default
    static const String baseUrl = 'http://localhost:8080';

    static Future<List<String>> getDistros() async {
        // hit the route defined in the server code
        final response = await http.get(Uri.parse('$baseUrl/api/distros'));
    }
}
```



```

    if (response.statusCode == 200) {
      // if OK get a list of the distro names
      final data = json.decode(response.body);
      final distros = data['distros'] as List;
      return distros.map((d) => d['name'] as
String).toList();
    } else {
      throw Exception('Failed to load distros');
    }
  }
}

```

In `main.dart`, I defined a function to call the API service and the `getDistros()` method to get the distro names:

```

Future<void> _loadDistros() async {
  ...
  try {
    final distros = await ApiService.getDistros();
    setState(() {
      _distros.clear();
      _distros.addAll(distros);
    });
  } catch (e) {
    ...
  }
}

```

On initialisation of the GUI, that function was fired:

```

@override
void initState() {
  super.initState();
  _loadDistros(); // <= function call

  // Later, I added a refresh to keep the list up-to-date
  _refreshTimer = Timer.periodic(const Duration(seconds: 5),
(timer) {
    _loadDistros();
  });
}

```

The data with the distro names was then used to populate Flutter's `ListView` widget, which was nested inside a `Builder` widget and `Expanded` widget:

```

Expanded(
  child: _distros.isEmpty
    // handle empty distro list
    ? const Center(child: Text("No distros loaded"))
    // apply logic before creating list UI
    : Builder(
      builder: (context) {

```

```

        // Copy and sort the distro list
        final sortedDistros = List<Map<String,
dynamic>>.from(_distros);
        sortedDistros.sort((a, b) {
            final aName = a['name'] as String;
            final bName = b['name'] as String;

            if (aName.toLowerCase() ==
_defaultDistro.toLowerCase()) return -1;
            if (bName.toLowerCase() ==
_defaultDistro.toLowerCase()) return 1;

            return
aName.toLowerCase().compareTo(bName.toLowerCase());
        });

        // Build the scrollable list
        return ListView.builder(
            itemCount: sortedDistros.length,
            itemBuilder: (context, index) {

                // Get one distro from the list
                final distroData = sortedDistros[index];
                final distro = distroData['name'];
                ...

                // Convert it into a UI item
                return Card(
                    ...
                    child: ListTile(
                        ...
                        title: Row(
                            children: [
                                Text(
                                    distro,
                                    ...
                                ),
                            ],
                        );
                    },
                );
            },
        ),
    ),
)

```

Everything is a widget in Flutter, and UIs consist of nested widgets, in this case:

```

Expanded
└─ Builder
   └─ ListView.builder
      └─ Card
         └─ ListTile
            └─ Row
               └─ Text

```

4.3.10. Publish docs

[Commit 8389ded](#) (add Sphinx docs)

The Sphinx docs were hosted on Read the Docs (RTD).

This simply involved logging in to RTD and connecting to the GitHub repo.

A `.readthedocs.yaml` file also needed to be added to the `/docs/` directory.

[Commit 30c4732](#) (RTD integration)

RTD provides numerous features, such as automated build previews in PRs.

Screenshot of the documentation homepage [6]:

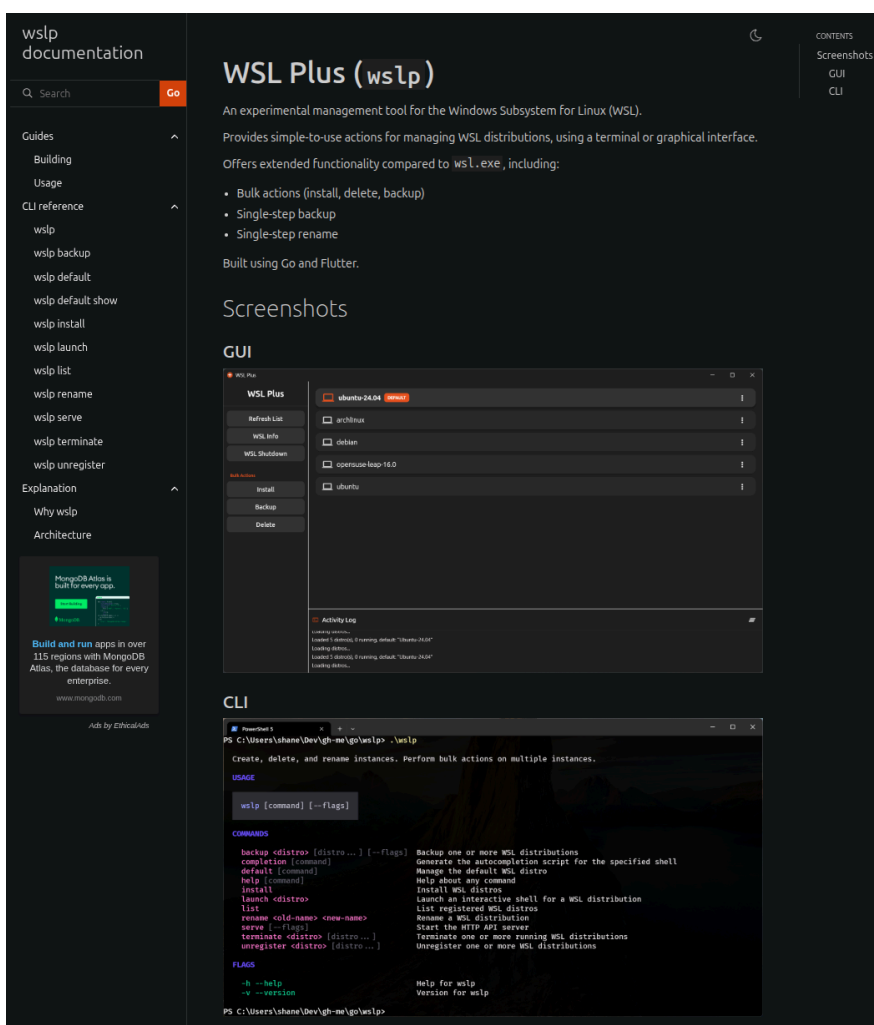


Figure 13: Documentation homepage on Read the Docs

4.3.11. Polish GUI

[Commit 3675a8c](#) (Add Yaru theming)

Yaru is the theme used to create Canonical's Flutter UIs.

This was added to the Flutter app:

```
// main.dart
...
import 'package:yaru/yaru.dart';

void main() {
  ...

  class MainApp extends StatelessWidget {
    ...

    @override
    Widget build(BuildContext context) {
      return YaruTheme(
        builder: (context, yaru, child) {
          return MaterialApp(
            theme: yaru.theme,
            darkTheme: yaru.darkTheme,
            ...
          );
        },
      );
    }
  }
}
```

Screenshot of the GUI:

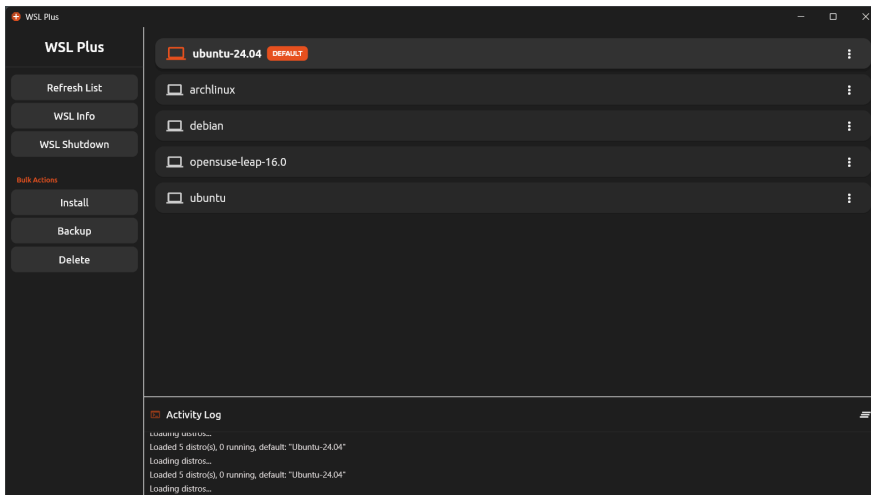


Figure 14: wslp GUI in dark mode

4.3.12. Add build/run scripts

[Commit 6c68ec1](#) (add convenience scripts)

The purpose of this project was not to build a production application.

The fact that it was a native Windows application also complicated packaging. For example, MSIX packages require signing through the Microsoft Store.

To make it easier for people to build and use the project, I added two batch scripts:

```
build.bat
```

```

@echo off
echo Building WSL Plus...
echo.

echo [1/2] Building wslp.exe (Go backend)...
go build -o wslp.exe
if %errorlevel% neq 0 (
    echo ERROR: Failed to build wslp.exe
    exit /b 1
)
echo ✓ wslp.exe built successfully
echo.

echo [2/2] Building GUI (Flutter)...
cd gui
call flutter build windows --release
if %errorlevel% neq 0 (
    echo ERROR: Failed to build GUI
    cd ..
    exit /b 1
)
cd ..
echo ✓ GUI built successfully
echo.

echo =====
echo Build complete!
echo.
echo To use WSL Plus:
echo   - CLI: Run 'wslp.exe' from the command line
echo   - GUI: Run 'rungui.bat'
echo =====
pause

```

The rungui script started the server on port 8080 and also ran the application.

rungui.bat

```

@echo off
echo Starting WSL Plus GUI...

REM Start the server in the background
start /B wslp.exe serve

REM Wait a moment for server to start
timeout /t 2 /nobreak >nul

REM Launch the GUI
start "" "gui\build\windows\x64\runner\Release\gui.exe"

echo WSL Plus is running.
echo Close this window to stop the server.
echo.

REM Keep the server running until user closes this window

```

```
pause >nul
```

```
REM Kill the server process when done  
taskkill /F /IM wslp.exe >nul 2>&1
```

4.4. Feature summary

4.4.1. Implemented features (CLI)

- `wsl backup` : Back up one or more distros
- `wslp copy` : Copies a distro with a new name
- `wsl default show` : Show the default distro
- `wslp install` : Installs one or more distros
- `wslp launch` : Launches an instance
- `wslp list` : Displays all installed WSL instances
- `wslp rename` : Renames a distro
- `wslp server` : Start server (backend for GUI)
- `wslp terminate` : Stop one or more instances
- `wslp unregister` : Removes one or more distros

4.4.1.1. Not implemented in CLI

- `wsl default change` : Change the default distro

4.4.2. Implemented features (GUI)

- Dashboard overview of instances
- Backup action
- Status indicator for default distro
- Change the default distro
- Install action
- Launch action
- Rename action
- Terminate action
- Unregister action
- Telemetry toggle (Ubuntu only)
- Global WSL commands (e.g., shutdown)

4.5. Project documentation

The documentation [6] was built using Sphinx, a Python-based static site generator. Documentation was written using Markdown. The Canonical starter pack was used to quickly bootstrap the documentation site.

The documentation was then customised to include autogenerated CLI documentation [4]. Cobra has built-in support for autogenerated docs. I used the official documentation [1] and to develop the implementation.

I then integrated a call to the CLI documentation generator to the `make/batch` file, so that each time the documentation was built/previewed, the CLI documentation would be refreshed.

4.5.1. Autogeneration code

This was the implementation (truncated). There was some redundancy, as it handled other docs formats that would not be needed.

```
package main

import (
    ...
    "github.com/spf13/cobra/doc"
    "wslp/cmd"
)

func main() {
    out := flag.String("out", "../../docs/reference/", "output
directory")
    format := flag.String("format", "markdown", "markdown|man|
rest")
    front := flag.Bool("frontmatter", false, "prepend simple
YAML front matter to markdown")
    flag.Parse()

    if err := os.MkdirAll(*out, 0o750); err != nil {
        log.Fatal(err)
    }

    rootCmd := cmd.RootCmd
    rootCmd.DisableAutoGenTag = true // stable, reproducible
files (no timestamp footer)

    switch *format {
    case "markdown":
        if *front {
            prep := func(filename string) string {
                base := filepath.Base(filename)
                name := strings.TrimSuffix(base, filepath.Ext(base))
                title := strings.ReplaceAll(name, "_", " ")
                return fmt.Sprintf("---\ntitle: %q\nslug:
%q\ndescription: \"CLI reference for %s\"\n---\n\n", title,
name, title)
            }
            link := func(name string) string { return
strings.ToLower(name) }
            if err := doc.GenMarkdownTreeCustom(rootCmd, *out,
prep, link); err != nil {
                log.Fatal(err)
            }
        } else {
            if err := doc.GenMarkdownTree(rootCmd, *out); err !=
nil {
                log.Fatal(err)
            }
        }
        ...
    default:
        log.Fatalf("unknown format: %s", *format)
    }
}
```

```
}  
}
```

4.6. Deployment and usage

`wslp` was distributed as a standalone binary.

A `.zip` of the CLI and GUI was also provided.

Users are advised to build from source.

Instructions for building/usage are provided in the README and official documentation [7].

5. Testing and Evaluation

5.1. General evaluation

Each feature was manually tested during its implementation.

Usually, this involved comparing the result of an implemented feature with some independently-verifiable state on the system.

Below, I include some examples of this verification:

- List: the app lists x, y, z. Does the output of `wsl.exe -l -v` also list x, y, z?
- Install: the app has installed x and y. Can I run `wsl.exe -d x` and `wsl.exe -d y` to run those distros?
- Rename: the app indicates a rename from x => z. Is that rename reflected in the Windows registry?

Occasionally, I would also verify in the opposite direction. For example, if I registered and launched a distro using `wsl.exe` commands would those distros then show with the correct statuses in the GUI interface?

5.2. Testing approach

The Go language has built in tooling for tests.

5.2.1. Approaches

The testing approach mostly consisted of unit tests.

To ensure separation of concerns, the business logic was separated from the command definitions into their own functions.

These functions were then tested in isolation using the native Go testing package:

```
go test
```

The aim was to maximize test coverage, which can be measured with:

```
go test -cover
```


5.2.2. Scope and constraints

The goal was to cover each command.

This involved writing a `<cmd>_testing.go` file to accompany each `<cmd>.go`.

Conservatively, that was 5-6 tests.

Below is an example test for the list command, which essentially checks for the existence of expected strings in the command output. This case was relatively simple, because it did not modify the host environment.

```
func TestListCommand(t *testing.T) {
    t.Run("command metadata", func(t *testing.T) {
        if RootCmd == nil {
            t.Fatal("RootCmd is nil")
        }

        listCmd, _, err := RootCmd.Find([]string{"list"})
        if err != nil {
            t.Fatalf("list command not found: %v", err)
        }

        if listCmd.Use != "list" {
            t.Errorf("expected Use='list', got '%s'", listCmd.Use)
        }

        // ... omitting for brevity
    })

    t.Run("command output", func(t *testing.T) {
        skipIfNotWindows(t)

        buf := new(bytes.Buffer)
        RootCmd.SetOut(buf)
        RootCmd.SetErr(buf)
        RootCmd.SetArgs([]string{"list"})

        err := RootCmd.Execute()
        if err != nil {
            t.Fatalf("command failed: %v", err)
        }

        output := buf.String()

        // Check for expected output patterns
        expectedPhrases := []string{
            "Finding registered distros",
            "distros are registered",
        }

        for _, phrase := range expectedPhrases {
            if !strings.Contains(output, phrase) {
                t.Errorf("expected output to contain '%s', got: %s", phrase, output)
            }
        }
    })
}
```

```

\n%s", phrase, output)
    }
}
})
}

```

Certain commands, such as those involving installation and registration, make significant modifications to the host environment.

Unit tests of these commands will need to use mocking:

```

// Example from current implementation of install.go
//...
// We don't want to actually install something during tests
type Installer interface {
    Install(ctx context.Context, distro string) error
}
//...

```

Effective integration tests would have tested the actual side-effects, however. This was probably best handled in a CI pipeline. It was possible to test within WSL virtual environments using GitHub actions. Thus, we could have created a WSL environment in the cloud, checked WSL was installed, installed the CLI tool, run a series of commands and checked if they worked as expected. However, this was outside of scope due to its complexity. Integration testing of any GUI application was certainly out of scope.

Another limitation affecting this project was that it needed to run on a Windows machine. I was primarily a Linux user, but used Windows for testing. A secondary advantage of unit tests was that I could test the logic of the app on Linux.

5.2.3. Interface-based mocking

The core challenge when testing this application is that the WSL operations it performs — installing distros, unregistering them, setting defaults — have side effects on the host system. In addition, calls to the WSL API only work on Windows systems.

Running tests that called `gowsl` directly would risk modifying the user's WSL environment, so it was avoided.

The solution was to use interfaces and dependency injection throughout the `internal/wsl` package, so that tests could substitute in mock implementations, without touching WSL at all.

Each operation in `internal/wsl` was backed by a small interface. For example, `list.go` defined a `Lister` interface:

```

type Lister interface {
    List(ctx context.Context) ([]string, error)
}

```

For the app itself, a `RealLister` struct implemented this interface using `gowsl`. For tests, a `mockLister` struct implemented the same interface and returned whatever data the test specified:

```
type mockLister struct {
    names      []string
    shouldFail bool
}

func (m *mockLister) List(ctx context.Context) ([]string,
error) {
    if m.shouldFail {
        return nil, errors.New("mock error")
    }
    return m.names, nil
}
```

This pattern was repeated for each operation: `Unregisterer`, `DefaultGetter`, `DefaultSetter`, `Installer`, and so on.

5.2.4. Two layers of tests

Tests are written at two levels that mirror the package structure.

`internal/wsl/` — *logic layer*

These tests verified that the core functions behaved correctly given controlled inputs but no actual WSL calls were made:

```
func TestListDistros(t *testing.T) {
    t.Run("returns registered distros", func(t *testing.T) {
        mock := &mockLister{names: []string{"Ubuntu",
"Debian"}}

        result, err := ListDistros(context.Background(),
mock)
        if err != nil {
            t.Fatalf("unexpected error: %v", err)
        }

        if len(result) != 2 {
            t.Errorf("expected 2 distros, got %d",
len(result))
        }
    })
    ...
}
```

`cmd/` — *command layer*

These tests verified that commands produced the correct output for a given input. They injected a mock through the same interface, and captured printed output:

```
func TestListCommand(t *testing.T) {
    t.Run("prints distros and count", func(t *testing.T) {
```

```

    mock := &mockLister{names: []string{"Ubuntu",
"Debian"}}
    out := new(bytes.Buffer)

    err := ListDistros(context.Background(), mock, out)
    if err != nil {
        t.Fatalf("unexpected error: %v", err)
    }

    output := out.String()
    for _, phrase := range []string{"2 distros are
registered", "Ubuntu", "Debian"} {
        if !strings.Contains(output, phrase) {
            t.Errorf("expected output to contain %q",
phrase)
        }
    }
})
}

```

5.2.5. Running the tests

```
go test ./...
```

I also used Codecov to create a testing pipeline on GitHub on PR or push:

<https://about.codecov.io/>

However, the implementation of tests did not keep pace with development, so coverage is low at 14%.

5.3. Evidence

Evidence of the utility of the application is included in the official documentation, which shows side-by-side comparisons of actions with and without the app [8].

Evidence functionality are shown using:

- Code coverage badge
- Documentation build status
- Screenshots of the app

Specific experiments were performed to test performance of concurrent installations.

For example:

Tested with: Ubuntu-24.04, archlinux, Debian

Mode	Run 1	Run 2	Run 3	Average
Concurrent	108.13	90.53	107.49	102
Sequential	155.25	163.66	143.49	154

Result: 34% reduction in installation time

These tests were run using PowerShell:

```
Measure-Command { wslp.exe install <distro-1> <distro-2>  
<distro-n> --experimental-concurrent }
```

The results are included in the documentation [\[5\]](#).

5.4. Evaluation

Project success was defined against the following criteria:

5.4.1. The tool broadly matches core functionality of wsl.exe

The tool could be used to list and register instances.

5.4.2. The tool extends the functionality of wsl.exe

The tool could be used to perform actions like registration in bulk.

5.4.3. The tool showcases distro-specific actions

The tool could be used to perform actions unique to distros, such as Ubuntu telemetry.

5.4.4. The tool integrates with a GUI front-end (at least partially)

Integration with a Flutter GUI was demonstrated.

5.4.5. Limitations

- Test coverage is currently poor
- Testing of GUI is not implemented
- Integration tests not implemented
- Incomplete feature parity between CLI and GUI
- Awkward installation process for users
- Flutter code needs to be refactored (monolithic)

6. Reflection

6.1. Challenges encountered

6.1.1. Scoping

Occasionally, there were planned tasks in the backlog that were considered out of scope and were marked “rejected”. If I had more time, these would be picked up in future iterations:

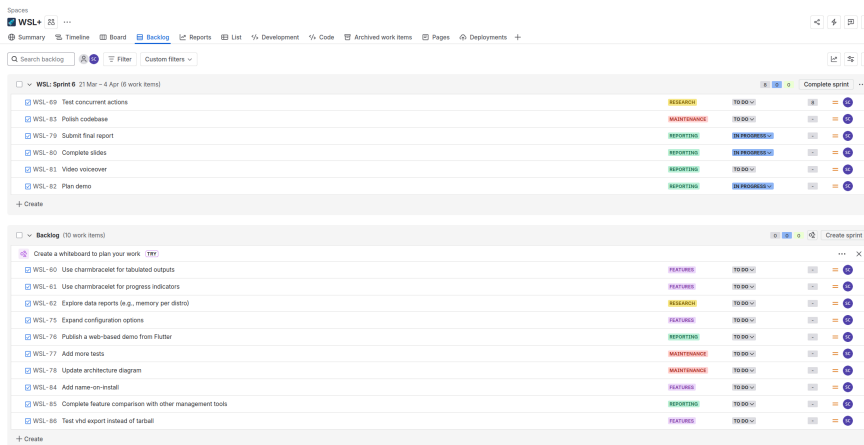


Figure 15: Jira backlog showing rejected tasks

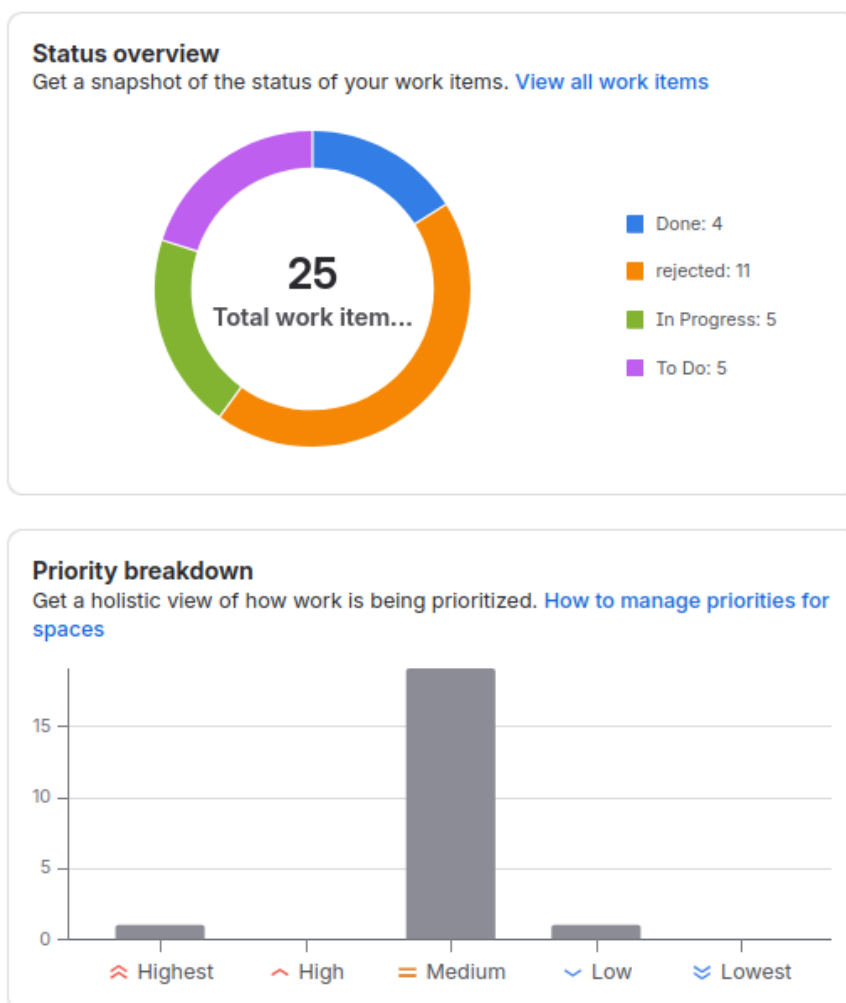


Figure 16: Jira project summary

6.1.2. Install command not working with classic distros

Recently, Microsoft started providing WSL distros in a new format. This was an important change, because this new tarball format obviates the need for the Microsoft Store accessible. This was a sticking point for enterprise users of WSL, where access to the Store is often restricted.

In the initial implementation, `wslp install <distro>` resulted in two outcomes:

1. Classic distro: successful installation is logged but distro cannot be launched
2. Modern distro: successful installation and the distro can be launched

To the user, (1) presents as a silent failure. What's happening is that the old distro format when installed usually triggers an interactive registration. The GoWSL library wraps `wsl.exe --install` for its install action but it also passes the `--no-launch` flag, which blocks the interactive installation. This may often be appropriate behavior, as GoWSL could be run in a CI pipeline, where an interactive prompt is not desirable.

To solve this problem, I considered the following solutions.

6.1.2.1. 1. implement a check for distro registration

A user would pass a string to identify the distro for installation.

That string could then be used to check if the distro was registered.

If the distro was not registered, a registration function could be called on that distro.

6.1.2.2. 2. log out information for the user

If a user installs a classic distro, they should be informed if the behavior deviates from the modern install.

6.1.2.3. 3. feedback to upstream

I contacted a maintainer of GoWSL and raised the issue.

They indicated that a possible solution could be placing `--no-launch` behind a flag.

They acknowledged that the semantics of the install could be confusing given that the behavior could vary depending on the distro.

6.1.2.4. 4. solution

The `IsRegistered()` method was used to check if an installed distro was registered and report distro state to the user:

```
for _, distro := range args {
    fmt.Printf("Installing %s...\n", distro)
    if err := gowsl.Install(context.Background(), distro);
err != nil {
    fmt.Printf("Error installing %s: %v\n", distro, err)
    continue
}
    fmt.Printf("Successfully installed %s\n", distro)

    d := gowsl.NewDistro(context.Background(), distro)
    registered, err := d.IsRegistered()
```

```
if err != nil {
    fmt.Printf("Error checking registration: %v\n", err)
} else if registered {
    fmt.Printf("%s was downloaded in the modern format and
is already registered\n", distro)
    fmt.Printf("Launch with wsl -d %s", distro)
} else {
    fmt.Printf("%s was downloaded in the classic format and
must be registered\n", distro)
    fmt.Printf("Register with wsl --register %s", distro)
}
}
```

6.2. Possible future improvements

6.2.1. Improve test coverage

The test coverage was about 22%. This could obviously have been higher.

The tests were time-consuming to implement, because of the need to do mocking.

Accelerating the writing of these tests seems like a good future task for an LLM.

6.2.2. Make available as a package

Currently, a user is advised to build the tool and run it with a script. A zip of the CLI and GUI with a batch script for running the app is also provided, but users still need to extract the files and run a script, which is not ideal.

This is normal for an open-source project, especially one in an early stage of development. In future, it should be packaged for distribution.

This would require certification through the Microsoft Store, which was considered overkill for a project at this stage of development.

6.2.3. Refactor the Flutter code

While the Go codebase had a clear separation-of-concerns, dividing specialized files into `/internal/` and `/cmd/`, the Flutter code largely existed in a monolithic `main.dart` that was difficult to read and maintain.

Similarly to the Go code, the Dart code should be refactored into specialized chunks.

6.2.4. Improve feature parity

Certain actions are possible in the GUI and not in the CLI, and vice versa; for example, it is possible to change the default distro in the GUI but this is not implemented in the CLI. Similarly, concurrent installation is allowed by the CLI but not the GUI. This results in an inconsistent user experience.

6.2.5. Better configuration options

Mode	Run 1	Run 2	Run 3	Average
Concurrent	108.13	90.53	107.49	102
Sequential	155.25	163.66	143.49	154

The Viper library is used for some basic configurations but they have not been adequately tested.

For example, increasing the number of parallel installations possible during concurrent installs didn't seem to work.

6.2.6. Better error messages

Some things fail silently.

For example, if you try to concurrently install three distros, and the third is misspelled, it will install the two with correct names and provide no indication that the third distro was not installed.

7. Acknowledgements

Early discussions related to a WSL management tool were had with Carlos Nihelton, an engineer at Canonical.

7.1. AI and LLM usage

7.1.1. Decoding output from Windows terminal

An LLM was used to implement `decodeUTF16` in `internal/wsl/available.go`. Windows Terminal outputs text as UTF-16 instead of the standard UTF-8, which was causing issues when consuming the data in my code.

The function converts that output into a normal string that Go can work with. This was an annoying problem that got in the way of adding basic functionality when implementing the dashboard, and it was not an interesting problem to solve.

`GetAvailableDistros` subprocesses out to `wsl --list --online` to get the distros that are available online. CLI tools on Windows output UTF-16-encoded text.

When Go captures the output, it gets raw UTF-16 bytes, but Go generally expects UTF-8. Without decoding, I was getting blank output (erroneously).

This is contrast with `list.go`. There, registered distros are listed using the `gowsl` library, which handles the encoding internally. But there's no `gowsl` API for listing distros available online that are not yet installed, so `available.go` calls `wsl --list --online` directly.

7.1.1.1. Record of prompt and response

The free version of Claude from Anthropic was used [2].

I'm writing a Go program on Windows that runs `wsl --list --online` using `exec.Command`. When I try to print the output, I get blank output — it. I think the output might be UTF-16. How do I write a function in Go that converts the UTF-16 LE into a string that Go can handle?

— Prompt

The issue is that `wsl --list --online` (like many Windows console tools) writes UTF-16 LE to stdout. Go's `cmd.Output()` gives you the raw bytes, so you need to UTF-16 LE stores each code unit as two bytes, little-endian. You can decode it manually using Go's `unicode/utf16` and `unicode/utf8` packages: `<CODE>`.

— Response

7.1.2. Hotfix commit messages

A small number of commits were made directly to main, often to the `README`, and sometimes using the GitHub graphical interface.

Lately, when committing directly through GitHub, GitHub creates an AI-generated commit message.

These are generally quite poor in my experience, and lose a lot of context when describing code commits.

However, they are generally sufficient for minor copy edits to a `README`, for example.

I didn't edit a handful of these suggestions because I would have produced descriptions that were similar or identical.

8. Appendix (confidential)

I am a technical author at Canonical, the maintainer of Ubuntu. I work across multiple teams, including the WSL team.

This appendix provides some additional context for the project that, while not under an NDA, is not something I wish to publicize.

Needless to say, please do not circulate this appendix.

8.1. Corporate and product context

The background for the project is my work as a technical author in the Ubuntu on WSL team at Canonical. Recently, we released a GUI application called Ubuntu Pro for WSL. While a user can install and open this app on their Windows machine, it is primarily aimed at system administrators.

The app enables system administrators to manage fleets of WSL instances running across multiple Windows hosts, ensuring that they are compliant and secure. For example, it enables a sys admin to remotely apply security patches to Ubuntu instances running on WSL

within hundreds of Windows machines in which the Pro for WSL app is installed.

At the same time, a regular user is able to install and use the app for free. It does have some utility in that circumstance, as it allows the attachment of a free subscription that improves security updates. However, it is still not a very useful or exciting app for most WSL users.

The GUI does not offer much to the user directly, it simply reports to the user that their subscription is active on their machine. It is more empowering to the sys admin in an organisational setting.

I suggested to my team that it would be nice if the app offered more to the individual user. For example, it would give them some ability to manager their WSL instances, not merely have their instances managed by someone else. The business case for this is that it might improve word-of-mouth about the app and encourage more organizations to purchase subscriptions. After all, they have developers who use WSL, and if it helps the developers they would want it too. However, we also produce a lot of open-source software, and it would also be good to simply give back to the community.

It never seemed like we had the time to pursue this. It also seemed like it might be more appropriate as a new, standalone app, which would require more capacity and resources, which we didn't have.

8.2. Early implementation discussions within my team

During these discussions with the team, I proposed using the GoWSL library to create a CLI tool that extends the functionality of wsl.exe. GoWSL is maintained by my team but it is used primarily as middleware for building CI pipelines; for example, it can be used to automate the testing of apps that target WSL in a GitHub-action-based test suite.

My thinking was that using the library would keep the code for the app clean, and consistent with our practices, but would also provide more opportunities to battle-test the library, and communicate that it can be used for more than CI pipelines, thereby attracting more contributors.

The team said “that might be interesting”. That was all! We had other things to do.

8.3. Capacity issues mean we cannot develop something like this

The engineers on the team have not had capacity to explore a full implementation further. There is also uncertainty about whether we should go this direction at all. Microsoft, for example, could develop something like this in the future, and we don't want to duplicate their efforts.

I am a technical author on the team, not an engineer, so while I can influence decisions about product and design, it is not my role to write the code.

For these and other reasons, I did not make this a “work project”. The value of the project for me, in addition to what is outlined in the main report, includes:

- Developing a prototype to demo for my team to convince them of the idea’s merits
- Show how the GoWSL library can be used in a user-facing app
- Learn more about the tooling used for the Pro for WSL app (Go and Flutter)

8.4. The impact if my project could be hyper specific

One of the features in the app is Ubuntu-specific. It is a toggle for enabling/disabling telemetry in new Ubuntu instances. It is foreseeable that a future iteration of the Pro for WSL app could implement that one feature alone. I can now refer to my implementation as an example. But more than that, I want to get across to the team the idea of Ubuntu-specific actions.

An engineer on my team expressed skepticism about the possibility of running concurrent installations of WSL distros. In this project, I demonstrated that it is possible and that it has efficiency benefits. That might not necessarily translate into a standalone product, but it could lead to feature discussions around WSL management. For example, Pro for WSL can install WSL distros remotely on employee computers — could it install multiple concurrently?

My role as a technical author is unusual. I manage documentation but also influence product and design. But that influence is often subtle!

8.5. Perspective from gowsl maintainer on semantics

One of the technical issues I encountered during the project was inconsistent handling of installation by the GoWSL library.

I asked a maintainer of the GoWSL project what he thought.

He indicated that there was a problem with how concepts like registration and installation were represented in the GoWSL library, but there are also limitations due to how the WSL API is designed.

[...] we’re hitting a difference in semantics between modern vs appx-based distros. Installing the appx ones mean installing the appx, not registering a new instance. With modern distros there’s nothing to install, only registering new instances makes sense. But `gowsl.Install` only subprocesses `wsl -install` so it does whatever WSL does at installation time. I need to overhaul those semantics, but they are quite tricky to get right if the underlying tooling doesn’t and we cannot override concepts. It means that

Install should stay as is, but we should create a new metaphor on top of it I think.

The expectation is that the legacy appx-based distros will be phased out soon, so this is likely a temporary problem.

This is an example of the kind of conversation I have had with the engineers in relation to my project. They have not been about how to code something, but more exploratory in nature.

This has been a good outcome. As you can see from the quote above, it has initiated discussions and planning.

Bibliography

- anon (no date) *Cobra User Documentation*. Available at: <https://cobra.dev/docs/>.
- Anthropic (2026) *Claude*. Available at: <https://claude.ai/>.
- Charm Bracelet (2024) *Fang - Wrapper for pretty Go CLIs*. Available at: <https://github.com/charmbracelet/fang>.
- edibotopic (no date) *wslp CLI ref docs*. Available at: <https://wslp.readthedocs.io/reference>.
- edibotopic (no date) *wslp concurrency page*. Available at: <https://wslp.readthedocs.io/explanation/concurrent-installs>.
- edibotopic (no date) *wslp docs*. Available at: <https://wslp.readthedocs.io/>.
- edibotopic (no date) *wslp guides*. Available at: <https://wslp.readthedocs.io/guides>.
- edibotopic (no date) *wslp justification*. Available at: <https://wslp.readthedocs.io/explanation/why>.
- Google (2024) *The Go Programming Language*. Available at: <https://go.dev/>.
- Microsoft (2024) “Windows Subsystem for Linux Architecture”. Available at: <https://learn.microsoft.com/en-us/windows/wsl/architecture>.
- Microsoft (2024) *WSL User Documentation*. Available at: <https://learn.microsoft.com/en-us/windows/wsl/>.
- Paul Hammant (2024) *Trunk Based Development*. Available at: <https://trunkbaseddevelopment.com/>.
- Prasad, A. *et al.* (2022) *Command Line Interface Guidelines*. Available at: <https://clig.dev/>.
- Steve Francia (2024) *Viper - Go configuration with fangs*. Available at: <https://github.com/spf13/viper>.
- Ubuntu (2024) *GoWSL - Go bindings for WSL API*. Available at: <https://github.com/ubuntu/gowsl>.

List of figures

Figure 1	System architecture	8
Figure 2	Model of GUI implementation	12
Figure 3	Flat command hierarchy	13
Figure 4	Layered command hierarchy	13
Figure 5	Jira project timeline	16
Figure 6	Pull request workflow on GitHub	16
Figure 7	Issue tracking on GitHub	17
Figure 8	Updated project timeline before final sprint	17
Figure 9	Cumulative progress during the project	17
Figure 10	Burndown chart for sprint 3	18
Figure 11	CLI appearance after Fang integration	24
Figure 12	Auto-generated CLI reference page	28
Figure 13	Documentation homepage on Read the Docs	35
Figure 14	wslp GUI in dark mode	36
Figure 15	Jira backlog showing rejected tasks	46
Figure 16	Jira project summary	46